

STAGE 2: Dart Intermediate (OOP, Collections, Async)

Flutter & Dart: From Zero to Expert — Stage Textbook 2 of 8

Goal: Master object-oriented Dart, work confidently with collections, and understand asynchronous programming — the three pillars that every Flutter app depends on.

Estimated time: 3-4 weeks

Prerequisite: Stage 1 (Dart Fundamentals)

2.1 Classes & Objects

```
class Person {  
  // Fields  
  String name;  
  int age;  
  
  // Constructor  
  Person(this.name, this.age);  
  
  // Named constructor  
  Person.guest() : name = 'Guest', age = 0;  
  
  // Method  
  void introduce() {  
    print('Hi, I am $name and I am $age years old.');  }  
}  
  
void main() {  
  var alice = Person('Alice', 30);  
  alice.introduce();  
  
  var guest = Person.guest();  
  guest.introduce();  
}
```

Key rules:

- Every Dart class implicitly has a default constructor unless you define one.
 - Named constructors (`Person.guest()`) give you multiple ways to build an object.
 - Use `this.fieldName` in the constructor parameter list as shorthand for assignment.
-

2.2 Inheritance, Abstract Classes & Mixins

```

abstract class Animal {
    String get sound; // abstract getter, must be implemented

    void makeSound() {
        print('The animal says $sound');
    }
}

class Dog extends Animal {
    @override
    String get sound => 'Woof';
}

mixin Swimmer {
    void swim() => print('Swimming...');
}

class Duck extends Animal with Swimmer {
    @override
    String get sound => 'Quack';
}

void main() {
    Dog().makeSound(); // The animal says Woof
    var duck = Duck();
    duck.makeSound(); // The animal says Quack
    duck.swim(); // Swimming...
}

```

Key rules:

- **abstract class** cannot be instantiated directly; it defines a contract.
- **extends** gives single inheritance; **with** adds one or more mixins for shared behavior without a strict "is-a" relationship.
- **@override** is not required but is best practice for clarity and compiler checking.

2.3 Collections: List, Set, and Map

```

void main() {
    // List - ordered, allows duplicates
    List<String> fruits = ['apple', 'banana', 'apple'];
    fruits.add('cherry');
    print(fruits.length); // 4

    // Set - unordered, no duplicates
    Set<String> uniqueFruits = fruits.toSet();
    print(uniqueFruits); // {apple, banana, cherry}

    // Map - key-value pairs
    Map<String, int> prices = {
        'apple': 200,
        'banana': 100,
    };
    prices['cherry'] = 500;
    print(prices['apple']); // 200

    // Collection methods (functional style)
    var doubled = [1, 2, 3].map((n) => n * 2).toList(); // [2, 4, 6]
    var evens = [1, 2, 3, 4, 5].where((n) => n.isEven).toList(); // [2, 4]
    var total = [1, 2, 3].fold<int>(0, (sum, n) => sum + n); // 6
}

```

Key rules:

- Prefer `List`, `Set`, and `Map` literals over manual construction where possible.
 - `.map()`, `.where()`, `.fold()`, and `.reduce()` are the workhorses of Dart collection processing — Flutter widget code uses these constantly to build UI lists from data.
-

2.4 Generics

```
class Box<T> {
  T content;
  Box(this.content);

  void show() => print('Box contains: $content');
}

T firstElement<T>(List<T> list) => list[0];

void main() {
  var intBox = Box<int>(42);
  var stringBox = Box<String>('Hello');
  intBox.show();
  stringBox.show();

  print(firstElement<String>(['x', 'y', 'z'])); // x
}
```

Why it matters: Generics let you write one reusable class or function for many types, with full compile-time type safety. Flutter's `List<Widget>`, `Future<T>`, and `State<T>` all lean on generics.

2.5 Asynchronous Programming: Future & async/await

```
Future<String> fetchUserName() async {
  await Future.delayed(Duration(seconds: 2)); // simulate network delay
  return 'Alice';
}

void main() async {
  print('Fetching...');
  String name = await fetchUserName();
  print('Got: $name');

  // Handling errors
  try {
    await Future.delayed(Duration(seconds: 1));
    throw Exception('Something went wrong');
  } catch (e) {
    print('Caught error: $e');
  }
}
```

Key rules:

- `async` marks a function as asynchronous; it always returns a `Future`.
 - `await` pauses execution until the `Future` completes, without blocking the whole program.
 - Always wrap `await` calls that can fail in `try/catch`.
-

2.6 Streams (a quick first look)

```
Stream<int> countStream() async* {
  for (int i = 1; i <= 5; i++) {
    await Future.delayed(Duration(milliseconds: 500));
    yield i;
  }
}

void main() async {
  await for (final value in countStream()) {
    print('Tick: $value');
  }
}
```

Why it matters: A `Stream` is like a `Future` that can emit multiple values over time. Flutter's `StreamBuilder` widget consumes streams directly — this is the foundation for live data like chat messages or sensor readings.

2.7 Hands-On Project: Contact Book CLI

Build a command-line Dart program that:

1. Defines a `Contact` class with `name`, `phone`, and an optional `email` (nullable `String?`).
2. Stores contacts in a `List<Contact>`.
3. Implements functions to: add a contact, search by name (using `.where()`), and sort contacts alphabetically (using `.sort()`).
4. Simulates "loading contacts from a server" with an `async` function that uses `Future.delayed` before returning the initial contact list.
5. Uses a `Map<String, int>` to count how many contacts share each first letter of their name.

This project combines classes, collections, generics (implicitly, through `List<Contact>`), and `async` — everything from Stage 2.

2.8 Stage 2 Test

Question 1: What is the difference between `extends` and `with` (mixins) in Dart?

Question 2: Write a generic function `T maxOf<T extends num>(T a, T b)` that returns the larger of two numbers.

Question 3: Given `List<int> nums = [1, 2, 3, 4, 5, 6]`, write one line using `.where()` and `.map()` that returns a list of the squares of only the even numbers.

Question 4: What does the `await` keyword do, and why must it be used inside an `async` function?

Question 5: What is the difference between a `Future` and a `Stream`? Give a real-world example where you'd use each.

Passing score: Get all 5 correct before moving to Stage 3.